

PC10

1. 第十章总体：线性方程组的求解

这份 **PC10.pdf** 讲第三篇“并行数值算法”中的 **第十章：线性方程组的求解**。这一章主要讨论不同结构的线性方程组怎样并行求解，包括三角形方程组、三对角方程组、稠密线性方程组和稀疏线性方程组。

1.1 本章结构

第十章包括四节：

- 10.1 三角形方程组的求解
- 10.2 三对角方程组的求解
- 10.3 稠密线性方程组的求解
- 10.4 稀疏线性方程组的求解

这一章的主线是：

不同线性方程组的矩阵结构不同，并行化方法也不同。矩阵越规则、依赖越局部，越容易找到并行算法；矩阵依赖越强，越难直接并行。

2. 10.1 三角形方程组的求解

这一节先讲线性方程组的基本符号，然后讲上三角方程组的回代解法如何并行化。

2.1 10.1.1 基本术语

线性方程组一般写成：

$$\begin{aligned}a_{11} x_1 + a_{12} x_2 + \dots + a_{1n} x_n &= b_1 \\a_{21} x_1 + a_{22} x_2 + \dots + a_{2n} x_n &= b_2 \\&\dots \\a_{n1} x_1 + a_{n2} x_2 + \dots + a_{nn} x_n &= b_n\end{aligned}$$

矩阵形式是：

$$AX = b$$

其中：

- A 是 $n \times n$ 系数矩阵
- x 是未知向量
- b 是右端项向量

第 5 页给出了这个矩阵形式，把线性方程组统一写成 $AX=b$ 。后面所有方法，本质上都是围绕如何求 x 展开。

2.2 10.1.2 上三角方程组的求解

上三角方程组的系数矩阵形如：

$$\begin{aligned}&a_{11} \ a_{12} \ \dots \ a_{1n} \\0 \quad &a_{22} \ \dots \ a_{2n} \\0 \quad 0 \quad &\dots \ a_{3n} \\&\dots \\0 \quad 0 \quad &\dots \ a_{nn}\end{aligned}$$

它通常通过 **回代 back substitution** 求解。

串行回代算法从最后一个未知量开始：

```
xn = bn / ann
x_{n-1} = (b_{n-1} - a_{n-1,n}xn) / a_{n-1,n-1}
...
```

也就是先求后面的变量，再用它们更新前面的方程。

2.3 SISD 上的回代算法

第 7 页给出 SISD 上的回代算法：

```
for i = n downto 1 do
  xi = bi / aii
  for j = 1 to i-1 do
    bj = bj - aji xi
    aji = 0
  endfor
endfor
```

这里有两个层次：

外层 i ：必须从 n 到 1 顺序执行
内层 j ：对多个方程 b_j 的更新可以并行执行

也就是说，求 x_i 这一步有顺序依赖，但一旦 x_i 已经求出，用它去更新前面多个方程时，这些更新可以同时做。

第 7 页图中也用括号标出内层 j 循环是“可并行化”的部分。

2.4 SIMD-CREW 上的并行回代算法

第 8 页给出 SIMD-CREW 上的并行回代算法。

2.4.1 划分方式

课件采用：

p 个处理器，行循环带状划分

也就是处理器按循环方式负责不同行：

P_1 负责第 $1, 1+p, 1+2p, \dots$ 行
 P_j 负责第 $j, j+p, j+2p, \dots$ 行

第 8 页右图展示了这种行循环分配方式。

2.4.2 并行算法

伪代码是：

```
for i = n downto 1 do
  xi = bi / aii

  for all  $P_j$ , where  $1 \leq j \leq p$  do
    for k = j to i-1 step p do
      bk = bk - aki xi
      aki = 0
    endfor
  endfor
endfor
```

每次求出 x_i 后，所有处理器并行更新自己负责的行。

2.4.3 复杂度

课件给出：

```
p(n) = n
t(n) = n
```

含义是：

- 用 n 个处理器；
- 外层仍然有 n 次顺序步骤；
- 每一轮内的更新可以并行完成。

所以并行回代的时间仍是 $O(n)$ ，不能做到 $O(\log n)$ ，原因是变量之间存在明显的回代依赖链。

3. 10.2 三对角方程组的求解

三对角方程组是线性方程组中的一种特殊结构，系数矩阵只有主对角线、上对角线 and 下对角线非零。

形如：

```
g1 x1 + h1 x2 = b1
f2 x1 + g2 x2 + h2 x3 = b2
...
fi x_{i-1} + gi xi + hi x_{i+1} = bi
...
fn xn-1 + gn xn = bn
```

课件讲两种方法：

```
10.2.1 直接求解法
10.2.2 奇偶规约法
```

4. 10.2.1 直接求解法

4.1 Gauss 消去法

第 11 页讲三对角方程组的直接求解法，也就是 Gauss 消去法。

它仍然分为：

- ① 消元
- ② 回代

但课件特别指出：

由于三对角方程组的特殊性，一次消元或一次回代只涉及邻近一个方程，所以难以并行化。

第 11 页图中显示了三对角矩阵的消元过程：每次主元 a_{ii} 只影响相邻位置，待消为 0 的元素沿着对角线附近推进。这种局部递推结构导致强顺序依赖。

4.2 为什么难并行化

三对角直接消去的依赖类似：

第 i 步依赖第 $i-1$ 步的消元结果

因此很难让很多行同时消元。

这和普通稠密高斯消去不同：稠密矩阵每次选定主元后，可以同时更新大量剩余元素；三对角矩阵每步只影响邻近方程，并行空间较小。

5. 10.2.2 奇偶规约法

奇偶规约法是三对角方程组中更适合并行化的方法。

5.1 基本形式

第 13 页把三对角方程写成：

$$\begin{aligned} f_i x_{i-1} + g_i x_i + h_i x_{i+1} &= b_i \\ i &= 1 \sim n \\ f_1 &= h_n = 0 \end{aligned}$$

也就是每个未知量 x_i 只和左右邻居相关。

5.2 奇偶规约思想

奇偶规约的核心是：

利用相邻方程消去某一类编号的变量，例如先消去奇下标变量，只保留偶下标变量；问题规模减半后继续规约。

第 13 页写出三个相邻方程：

$$\begin{aligned} f_{2i-1} x_{2i-2} + g_{2i-1} x_{2i-1} + h_{2i-1} x_{2i} &= b_{2i-1} \\ f_{2i} x_{2i-1} + g_{2i} x_{2i} + h_{2i} x_{2i+1} &= b_{2i} \\ f_{2i+1} x_{2i} + g_{2i+1} x_{2i+1} + h_{2i+1} x_{2i+2} &= b_{2i+1} \end{aligned}$$

目标是消去中间偶方程中的 x_{2i-1} 和 x_{2i+1} ，让第 $2i$ 个方程变成：

$$\alpha_i x_{2i-2} + \beta_i x_{2i} + \gamma_i x_{2i+2} = \eta_i$$

这样原来相邻距离为 1 的三对角结构，变成了相邻距离为 2 的新三对角结构。

5.3 递归规约

第 14 页说明，重复这种消去过程，最后问题会缩小到只剩下：

case 1: 两个未知量
case 2: 三个未知量

例如最后可能得到：

$$\begin{aligned} g_1 x_1 + h_1 x_2 &= b_1 \\ f_2 x_1 + g_2 x_2 &= b_2 \end{aligned}$$

或者：

$$\begin{aligned} g_1 x_1 + h_1 x_2 &= b_1 \\ f_2 x_1 + g_2 x_2 + h_2 x_3 &= b_2 \\ f_3 x_2 + g_3 x_3 &= b_3 \end{aligned}$$

解出剩余未知量后，再回代求出其他变量。

5.4 并行化分析

课件第 14 页明确指出：

- ①、② 消去奇下标可以并行化；
- ③ 回代求解可以并行化。

为什么？

因为在同一轮规约中，各个偶编号方程的消去操作彼此独立，可以同时执行。

所以奇偶规约法相比直接 Gauss 消去，更适合三对角系统的并行求解。

6. 10.3 稠密线性方程组的求解

这一节讲稠密线性方程组，也就是矩阵 A 中大多数元素都非零的情况。

课件讲三种方法：

- 10.3.1 有回代的高斯消去法
- 10.3.2 无回代的高斯-约旦法
- 10.3.3 迭代求解的高斯-赛德尔法

7. 10.3.1 有回代的高斯消去法

7.1 基本原理

第 17 页讲有回代的高斯消去法。

求解过程分成两个阶段：

- ① 消元：把系数矩阵 A 化为上三角矩阵 T
- ② 回代：对 $TX = c$ 进行回代求解

消元过程中还可以使用选主元方法，提高数值稳定性。

第 17 页图 10.1 展示了消元过程：

主元行
待消为 0 的元素
已消为 0 的元素
待改变的元素
不变化的元素

每一步选定主元后，要用主元行去更新主元下方的许多行和列。

7.2 并行化分析

第 18 页指出：

消元和回代均可以并行化；
选主元也可以并行化；
处理器按行划分。

在消元阶段，一旦主元行确定，主元下方的多行更新可以并行进行。

例如第 i 步消元时，对所有 $j > i$ 的行：

$$Row_j = Row_j - factor \times Row_i$$

不同 j 的行更新可以同时执行。

第 18 页图中标出了“可并行化部分”，即主元下方大量待更新区域。

7.3 高斯消去法的特点

优点：

数值稳定性较好，尤其配合选主元
适合稠密矩阵
是直接法，通常能得到较准确结果

缺点：

计算量大，约 $O(n^3)$
通信和同步开销明显
主元选择会带来额外通信

8. 10.3.2 无回代的高斯-约旦法

8.1 串行算法原理

第 20 页讲高斯-约旦法。

它和普通高斯消去不同：

普通高斯消去：把 A 化成上三角矩阵，然后回代
高斯-约旦法：直接把增广矩阵 (A, b) 化成主对角矩阵

也就是说，高斯-约旦法通过初等行变换，把：

(A, b)

化成：

对角矩阵形式

然后直接求：

$x_j = a'_{\{j, n+1\}} / a'_{\{jj\}}$

不需要回代。

8.2 SIMD-CREW 上的并行算法

第 21 页给出 SIMD-CREW 上的并行算法。

8.2.1 处理器组织

使用：

$n^2 + n$ 个处理器

这些处理器排成：

$n \times (n+1)$

的矩阵。

处理器编号为：

$P_{ik}, i = 1 \sim n, k = 1 \sim n+1$

每个处理器负责增广矩阵中的一个元素。

8.2.2 消元并行化

第 j 次消元：

```
for j = 1 to n-1, each  $P_{ik}$  Par-do
   $P_{ij}(i \neq j)$ :  $a_{ij} \leftarrow 0$ 
   $P_{ik}(i \neq j, k = j+1 \sim n+1)$ :
     $a_{ik} \leftarrow a_{ik} - a_{jk}(a_{ij} / a_{jj})$ 
endfor
```

也就是说，每次以第 j 行为主元行，其他行的相关元素可以同时更新。

8.2.3 求解并行化

最后：

```
for each Pjj, j = 1~n Par-do:
    xj ← aj,n+1 / ajj
```

所有未知量可以同时求出。

8.3 时间、处理器数和成本

课件给出：

```
t(n) = O(n)
p(n) = O(n²)
c(n) = O(n³)
```

其中：

$$c(n) = t(n) \times p(n)$$

所以成本为 $O(n^3)$ 。

第 21 页最后问：“成本最优？”

下一页解释了这个问题。

8.4 成本最优性讨论

第 22 页讨论串行最优时间。

因为：

$$x = A^{-1}b$$

如果已有 A^{-1} ，计算 $A^{-1}b$ 需要：

$$O(n^2)$$

关键是求 A^{-1} 。

课件用分块矩阵求逆分析，设矩阵乘法复杂度为：

$$m(n) = O(n^\alpha), \quad 2 < \alpha < 2.5$$

则求逆也可达到：

$$O(n^\alpha)$$

因此串行算法的最优时间可以低于 $O(n^3)$ 。

所以高斯-约旦并行算法的成本：

$$O(n^3)$$

相对于最优串行算法不一定成本最优。

9. 10.3.3 迭代求解的高斯-赛德尔法

9.1 串行算法原理

第 24 页给出 Gauss-Seidel 迭代公式。

对于第 $k+1$ 次迭代：

$$\begin{aligned} x_i^{k+1} &= \\ & \frac{1}{a_{ii}} \cdot \left[b_i - \sum_{j < i} a_{ij} x_j^{k+1} - \sum_{j > i} a_{ij} x_j^k \right] \end{aligned}$$

注意这里有一个重要特征：

$j < i$ 的部分使用本轮新值 x_j^{k+1}
 $j > i$ 的部分使用上一轮旧值 x_j^k

这也是 Gauss-Seidel 比 Jacobi 更难并行化的根本原因。

9.2 收敛判据

课件给出的收敛条件是：

$$\sum_{i=1}^n |x_i^{k+1} - x_i^k| < c$$

如果满足给定误差允许值 c ，就认为迭代收敛。

9.3 为什么同步并行困难

Gauss-Seidel 的每次迭代需要使用本次迭代前面已经更新的值。

也就是说：

x_2^{k+1} 依赖 x_1^{k+1}
 x_3^{k+1} 依赖 x_1^{k+1} , x_2^{k+1}
...

这种顺序依赖让它很难得到同步并行算法。

因此课件给出的是 **MIMD 异步并行算法**。

9.4 MIMD 异步并行算法

第 25 页给出算法。

9.4.1 进程组织

使用 N 个处理器，且：

$$N \leq n$$

生成 n 个进程，每个进程计算 x 的一个分量。

9.4.2 算法过程

每个进程 i 重复执行：

```
old_i ← new_i

new_i ← (b_i
        - ∑_{k < i} a_ik old_k
        - ∑_{k > i} a_ik old_k) / a_ii
```

直到：

$$\sum |old_i - new_i| < c$$

最后：

```
xi ← new_i
```

9.5 关键理解

这个算法并不严格按照传统 Gauss-Seidel 的“使用本轮前面新值”来同步推进，而是异步读取其他进程当前可见的旧值或新值。

它牺牲了一些同步严格性，换取并行执行能力。

10. 10.4 稀疏线性方程组的求解

稀疏线性方程组指数组矩阵中大多数元素为 0。

这类问题常来自：

偏微分方程离散
有限差分
有限元
图结构问题
科学工程模拟

稀疏矩阵的特点是：

存储要利用稀疏性
计算只围绕非零元素
通信通常由矩阵的邻接结构决定

11. 10.4.1 线性方程组的并行化方法

11.1 选择算法的考虑因素

第 28 页列出选择线性方程组算法时要考虑的因素。

11.1.1 系数矩阵 A 的结构

不同结构适合不同方法：

dense: Gaussian elimination 等
sparse: iterative method
triangular: substitution, odd-even reduction
certain PDEs: multigrid 等

也就是说，矩阵结构直接决定算法选择。

11.1.2 计算精度要求

课件比较：

Gaussian elimination: more accurate, more expensive
Conjugate gradients: less accurate, less expensive

直接法通常更准确，但更贵；迭代法通常便宜，但精度和收敛性依赖问题性质。

11.1.3 计算环境要求

还要考虑：

architecture
available languages
compiler quality
libraries

也就是并行机结构、编程语言、编译器质量和是否有可用数值库。

11.2 求解方法的并行化分类

第 29 页把求解方法分为两大类。

11.2.1 直接解法的并行化

主要用于稠密线性方程组：

Gauss 消去法，包括选主元的 Gauss 消去法
Gauss-Jordan 消去法
LU 分解法

直接法通常计算量大，但稳定性和精度较好。

11.2.2 迭代法的并行化

用于稠密和稀疏线性方程组：

Jacobi
Gauss-Seidel, 可异步并行化
Jacobi OverRelaxation, JOR
Gauss-Seidel OverRelaxation, SOR
Conjugate Gradient

稀疏问题中迭代法尤其常用，因为它能避免对稀疏矩阵做大量填充和复杂消元。

12. 10.4.2 稀疏线性方程组的迭代解法

第 31 页列出几种常见迭代方法。

12.1 Jacobi 方法

Jacobi 迭代公式是：

$$x_i^{k+1} = (b_i - \sum_{j \neq i} a_{ij} x_j^k) / a_{ii}$$

特点：

本轮所有 x_i^{k+1} 都只依赖上一轮 x^k

因此 Jacobi 很容易同步并行化。

缺点是收敛速度通常比 Gauss-Seidel 慢。

12.2 Gauss-Seidel 方法

Gauss-Seidel 公式是：

$$x_i^{k+1} = (b_i - \sum_{j < i} a_{ij} x_j^{k+1} - \sum_{j > i} a_{ij} x_j^k) / a_{ii}$$

特点：

使用本轮已更新的新值

因此收敛通常较快，但并行化更困难。

12.3 JOR 方法

JOR 是 Jacobi OverRelaxation。

公式形式为：

$$\begin{aligned} x_i^{k+1} &= \\ & (1-\omega)x_i^k \\ & + \omega(b_i - \sum_{j \neq i} a_{ij} x_j^k) / a_{ii} \end{aligned}$$

其中 ω 是松弛因子。

它是在 Jacobi 基础上加入松弛调节。

12.4 SOR 方法

SOR 是 Successive OverRelaxation，也就是 Gauss-Seidel OverRelaxation。

公式为：

$$\begin{aligned} x_i^{k+1} &= \\ & (1-\omega)x_i^k \\ & + \omega(b_i \\ & \quad - \sum_{j < i} a_{ij} x_j^{k+1} \\ & \quad - \sum_{j > i} a_{ij} x_j^k) / a_{ii} \end{aligned}$$

它是在 Gauss-Seidel 基础上加入松弛因子。

适当选择 ω 可以加快收敛，但 ω 不合适也可能导致不收敛。

12.5 Conjugate Gradient

共轭梯度法适合某些稀疏、对称正定线性方程组。

课件这里只列出名称，没有展开公式。

它在大规模稀疏线性方程组中非常重要，因为它只需要矩阵-向量乘法、向量内积和向量更新，适合稀疏存储和并行计算。

13. 10.4.3 高斯-赛德尔迭代法的并行化

这一节重点讲由 PDE 离散产生的稀疏线性方程组，以及 Gauss-Seidel 如何通过结构重排实现并行。

13.1 Laplace 方程

第 33 页给出二维 Laplace 方程：

$$\begin{aligned} \partial^2 u(x,y) / \partial x^2 + \partial^2 u(x,y) / \partial y^2 &= g(x,y) \\ (x,y) \in D &= [0,1] \times [0,1] \end{aligned}$$

并要求满足边界条件：

$$u(x,y) = \phi(x,y)$$

图中显示了一个二维网格，边界点是已知边值，内部点是要求解的未知量。

13.2 五点格式离散

第 34 页假设：

$$g(x,y) = 0$$

用二阶中心差分近似二阶偏导。

对网格点 (i, j) ，得到：

$$u_{ij} = 1/4 \cdot (u_{i+1,j} + u_{i-1,j} + u_{i,j+1} + u_{i,j-1})$$

这就是典型的 **五点差分格式**。

也就是说，一个内部点只依赖上下左右四个邻居。

13.3 稀疏块三对角矩阵

第 34 页和第 35 页说明，五点格式离散后得到的系数矩阵 A 是稀疏的块三对角矩阵。

第 35 页图中：

- 左边是二维网格节点编号；
- 右边是矩阵非零元分布图；
- 红色和绿色小块表示非零位置。

这张图很重要，因为它说明：

PDE 离散后的矩阵不是随机稀疏，而是具有规则结构的稀疏矩阵。

这种结构可以被用来设计并行算法。

13.4 两种计算顺序

第 36 页讲 Gauss-Seidel 的两种串行计算顺序。

13.4.1 顺序 1

顺序 1 按行从左到右、从下到上计算。

这种顺序依赖很强：

当前点依赖左边和下边已经更新的点

所以课件说：

顺序 1 难以并行化

13.4.2 顺序 2

顺序 2 按斜对角线方向推进。

同一条斜对角线上的点之间没有直接依赖，可以小规模并行。

课件说：

顺序 2 可以小规模并行化

第 36 页右图中的斜线箭头就展示了这种 wavefront 思想。

13.5 红黑着色并行算法

第 37 页讲最重要的并行化方法：**红黑着色**。

13.5.1 基本思想

把二维网格点染成红黑两类，使得：

每个红点的上下左右邻居都是黑点
每个黑点的上下左右邻居都是红点

这类似棋盘格染色。

13.5.2 算法步骤

课件给出：

- ① 并行计算所有红点
- ② 并行计算所有黑点
- ③ 重复 ①、②，直到满足精度要求

13.5.3 为什么可以并行

五点格式中，每个点只依赖上下左右邻居。

如果所有红点的邻居都是黑点，那么红点之间互不直接依赖，所以可以同时更新。

同理，所有黑点也可以同时更新。

因此红黑着色把 Gauss-Seidel 原本的强顺序依赖，转化为两阶段并行：

- 红点同步更新
- 黑点同步更新

14. 本章方法总对比

问题类型	典型方法	并行化特点
上三角方程组	回代法	外层依赖强，内层更新可并行
三对角方程组	直接 Gauss 消去	只涉及邻近方程，难并行
三对角方程组	奇偶规约	每轮规模减半，适合并行
稠密线性方程组	高斯消去	行更新、选主元可并行
稠密线性方程组	高斯-约旦	无回代，元素级并行度高
稠密线性方程组	Gauss-Seidel	同步并行困难，可异步并行
稀疏线性方程组	Jacobi	最容易并行
稀疏线性方程组	Gauss-Seidel	依赖较强，可用红黑着色
稀疏线性方程组	SOR / CG	常用于大规模稀疏问题

15. 本章核心总结

第十章的核心是：

- 线性方程组结构不同
- 数据依赖不同
- 并行化方法不同

15.1 最重要的复习点

需要重点掌握：

1. 上三角回代中，外层顺序、内层可并行
2. 三对角直接法难并行，奇偶规约可并行
3. 高斯消去分为消元和回代，行更新可并行
4. 高斯-约旦法无回代，可用大量处理器并行消元
5. Gauss-Seidel 依赖本轮新值，因此同步并行困难
6. Jacobi 只依赖上一轮旧值，因此容易并行
7. PDE 五点格式产生稀疏块三对角矩阵
8. 红黑着色能把 Gauss-Seidel 转化为两阶段并行更新

一句话概括：

第十章讲的是线性方程组求解中的并行化策略：直接法主要并行化消元和更新，迭代法主要利用矩阵结构降低依赖；对于稀疏 PDE 问题，红黑着色是 Gauss-Seidel 并行化的关键技巧。

PC11

1. 第十一章总体：快速傅里叶变换

这份 PC11.pdf 讲第三篇“并行数值算法”中的 **第十一章：快速傅里叶变换**。这一章主要分两部分：先讲 DFT 和串行 FFT 的基本原理，再讲两个并行 FFT 算法：SIMD-MC² 上的 FFT 算法和 SIMD-BF 上的 FFT 算法。

1.1 本章结构

第十一章包括两节：

- 11.1 快速傅里叶变换
- 11.2 并行 FFT 算法

其中 11.1 又包括：

- 11.1.1 离散傅里叶变换 DFT
- 11.1.2 DFT 的顺序代码
- 11.1.3 串行 FFT 递归算法
- 11.1.4 串行 FFT 非递归算法

11.2 包括：

- 11.2.1 SIMD-MC² 上的 FFT 算法
- 11.2.2 SIMD-BF 上的 FFT 算法

这一章的主线是：

DFT 直接计算需要 $O(n^2)$ ，FFT 利用单位根性质和分治思想把复杂度降到 $O(n \log n)$ ，并行 FFT 则进一步利用处理器网络结构把计算和通信组织成蝶式模式。

2. 11.1 快速傅里叶变换 FFT

FFT 是 Fast Fourier Transform，快速傅里叶变换。

它不是一种新的变换，而是 **快速计算 DFT 的算法**。

2.1 DFT 与 FFT 的关系

DFT 是数学定义：

输入 $A = (a_0, a_1, \dots, a_{n-1})$
输出 $B = (b_0, b_1, \dots, b_{n-1})$

FFT 是计算方法：

用更少的运算求出同样的 B

直接 DFT：

$O(n^2)$

FFT：

$O(n \log n)$

3. 11.1.1 离散傅里叶变换 DFT

3.1 DFT 定义

第 5 页给出 DFT 的定义。

给定向量：

$$\mathbf{A} = (a_0, a_1, \dots, a_{n-1})^T$$

DFT 将它变换为：

$$\mathbf{B} = (b_0, b_1, \dots, b_{n-1})^T$$

其中：

$$b_j = \sum_{k=0}^{n-1} a_k \omega^{kj}, \quad 0 \leq j \leq n-1$$

这里：

$$\omega = e^{2\pi i/n}$$

是 n 次单位根， $i = \sqrt{-1}$ 。

3.2 矩阵形式

DFT 可以写成矩阵乘法：

$$\mathbf{B} = \mathbf{W} \mathbf{A}$$

其中 $\mathbf{W}$ 是由单位根幂组成的矩阵：

$$W[j, k] = \omega^{jk}$$

也就是：

$$\begin{bmatrix} b_0 \\ b_1 \\ \dots \\ b_{n-1} \end{bmatrix} = \begin{bmatrix} \omega^0 & \omega^0 & \dots & \omega^0 \\ \omega^0 & \omega^1 & \dots & \omega^{n-1} \\ \dots & \dots & \dots & \dots \\ \omega^0 & \omega^{n-1} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \dots \\ a_{n-1} \end{bmatrix}$$

这说明 DFT 本质上是一个特殊矩阵和向量的乘法。

4. 11.1.2 DFT 的顺序代码

第 7 页给出两段顺序代码。

4.1 代码 1：直接计算 $\omega^{k \cdot j}$

第一种代码是直接按照公式计算：

```
for j = 0 to n-1 do
  b[j] = 0
  for k = 0 to n-1 do
    b[j] = b[j] +  $\omega^{k \cdot j}$  a[k]
  end for
end for
```

这段代码最直观，但问题是：

每次内层循环都要计算 $\omega^{k \cdot j}$

如果直接求幂，会带来额外代价。

4.2 代码 2：递推计算 ω 的幂

第二种代码避免反复求幂：

```
w = w^0
for j = 0 to n-1 do
    b[j] = 0
    s = w^0
    for k = 0 to n-1 do
        b[j] = b[j] + s * a[k]
        s = s * w
    end for
    w = w * w
end for
```

这里的思想是：

对于固定 j ， $\omega^{\{kj\}}$ 可以递推得到

即：

$\omega^{\{0j\}}$, $\omega^{\{1j\}}$, $\omega^{\{2j\}}$, ...

不用每次重新计算幂。

4.3 复杂度

课件指出代码 2 的复杂度仍然是：

$O(n^2)$

因为有两层循环，每个 $b[j]$ 都要累加 n 项，总共 n^2 次主运算。

所以，虽然代码 2 减少了求幂开销，但没有改变 DFT 的数量级复杂度。

5. 11.1.3 串行 FFT 递归算法

FFT 的核心是把 DFT 拆成两个规模为 $n/2$ 的 DFT。

5.1 单位根性质

第 9 页先介绍单位根性质。

令：

$\omega = e^{\{2\pi i/n\}}$

再令：

$\omega_{\sim} = e^{\{2\pi i/(n/2)\}}$

也就是 $n/2$ 次单位根。

那么有：

$\omega_{\sim} = \omega^2$

此外，还会反复使用：

$\omega^n = 1$
 $\omega^{\{n/2\}} = -1$
 $\omega^{\{p+n\}} = \omega^p$

第 9 页右下角的圆图展示了 $n=8$ 时单位根在复平面上的位置，例如：

$$\begin{aligned}\omega^0 &= 1 \\ \omega^2 &= i \\ \omega^4 &= -1 \\ \omega^6 &= -i \\ \omega^8 &= 1\end{aligned}$$

这些周期性和对称性正是 FFT 能加速的数学基础。

5.2 输出向量按偶数项和奇数项拆分

第 9 页把输出向量 $\mathbf{B}$ 的偶数项和奇数项分别记为两个向量。

偶数下标输出：

$$(b_0, b_2, \dots, b_{n-2})^T$$

奇数下标输出：

$$(b_1, b_3, \dots, b_{n-1})^T$$

课件记成：

$$\begin{aligned}\mathbf{B}' &= \text{偶数项输出} \\ \mathbf{B}'' &= \text{奇数项输出}\end{aligned}$$

6. 偶数项的递归推导

6.1 偶数项公式

第 10 页推导偶数项。

对于偶数下标：

$$b'_{\{1\}} = b_{\{2\}}$$

根据 DFT 定义：

$$b_{\{2\}} = \sum_{k=0}^{n-1} a_k \omega^{2\{1\}k}$$

把 k 分成前半部分和后半部分：

$$\begin{aligned}k &= 0, 1, \dots, n/2-1 \\ k+n/2\end{aligned}$$

利用：

$$\omega^{2\{1\}(k+n/2)} = \omega^{2\{1\}k} \cdot \omega^{2\{1\}n} = \omega^{2\{1\}k}$$

所以可得：

$$\begin{aligned}b_{\{2\}} &= \sum_{k=0}^{n/2-1} \omega^{\sim\{1\}k} (a_k + a_{\{k+n/2\}})\end{aligned}$$

其中：

$$\omega^{\sim} = \omega^2$$

6.2 偶数项结论

因此，偶数项向量：

$$(b_0, b_2, \dots, b_{n-2})$$

是下面这个长度 $n/2$ 向量的 DFT：

$$\begin{aligned} &(a_0 + a_{n/2}, \\ &\quad a_1 + a_{n/2+1}, \\ &\quad \dots \\ &\quad a_{n/2-1} + a_{n-1}) \end{aligned}$$

也就是说：

$$\text{偶数输出} = \text{DFT}(n/2) \text{ of pairwise sums}$$

7. 奇数项的递归推导

7.1 奇数项公式

第 11 页推导奇数项。

对于奇数下标：

$$b'_{l+1} = b_{2l+1}$$

根据 DFT 定义：

$$b_{2l+1} = \sum_{k=0}^{n-1} a_k \omega^{(2l+1)k}$$

同样把 k 分成前半和后半：

$$k \text{ 和 } k+n/2$$

利用：

$$\begin{aligned} &\omega^{(2l+1)(k+n/2)} \\ &= \omega^{(2l+1)k} \cdot \omega^{(2l+1)n/2} \\ &= -\omega^{(2l+1)k} \end{aligned}$$

因为：

$$\begin{aligned} \omega^{ln} &= 1 \\ \omega^{n/2} &= -1 \end{aligned}$$

所以得到：

$$\begin{aligned} &b_{2l+1} \\ &= \sum_{k=0}^{n/2-1} \omega^{-l(2k+1)} \cdot \omega^{lk} \cdot (a_k - a_{k+n/2}) \end{aligned}$$

7.2 奇数项结论

因此，奇数项向量：

$$(b_1, b_3, \dots, b_{n-1})$$

是下面这个长度 $n/2$ 向量的 DFT：

```
((a0 - a_{n/2})\omega^0,
(a1 - a_{n/2+1})\omega^1,
...
(a_{n/2-1} - a_{n-1})\omega^{n/2-1})
```

也就是说：

奇数输出 = DFT(n/2) of weighted pairwise differences

8. 蝶式递归计算图

8.1 蝶式结构

第 12 页给出 FFT 的蝶式递归计算图。

输入被两两组合：

```
a0 与 a_{n/2}
a1 与 a_{n/2+1}
...
a_{n/2-1} 与 a_{n-1}
```

每一对产生两个量：

```
a_k + a_{k+n/2}
\omega^k(a_k - a_{k+n/2})
```

上半部分进入一个 $\text{DFT}(n/2)$ ，下半部分进入另一个 $\text{DFT}(n/2)$ 。

8.2 为什么叫蝶式

因为每一对输入通过加法和减法交叉连接，形状像蝴蝶的两翼。

一个基本蝶式计算可以抽象为：

```
u = x + y
v = (x - y) \cdot \omega^k
```

这个结构在 FFT 中反复出现。

9. $n = 8$ 的 FFT 蝶式展开图

第 13 页给出了 $n = 8$ 的完整展开图。

9.1 输入和输出顺序

输入是：

$a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7$

输出顺序在图中是：

$b_0, b_4, b_2, b_6, b_1, b_5, b_3, b_7$

这不是自然顺序，而是和 **位序反 bit reversal** 有关。

9.2 图中权因子

图中有不同的权因子：

```
1
\omega
\omega^2
\omega^3
```

这些权因子出现在下行分支上，对应蝶式中的：

$$(a_k - a_{k+n/2}) \cdot \omega^k$$

9.3 图的意义

这个展开图说明：

FFT 由 $\log n$ 层蝶式计算组成；
每层有 $n/2$ 个蝶式；
总计算量为 $O(n \log n)$ 。

10. SISD 上的 FFT 分治递归算法

第 14 页给出串行递归 FFT 算法 $\text{RFFT}(a, b)$ 。

10.1 算法输入输出

输入：

$$a = (a_0, a_1, \dots, a_{n-1})$$

输出：

$$B = (b_0, b_1, \dots, b_{n-1})$$

10.2 算法过程

伪代码为：

```
Procedure RFFT(a,b)
begin
  if n = 1 then
    b0 = a0
  else
    RFFT(a0, a2, ..., an-2, u0, u1, ..., u_{n/2-1})
    RFFT(a1, a3, ..., an-1, v0, v1, ..., v_{n/2-1})

    z = 1
    for j = 0 to n-1 do
      bj = u_{j mod n/2} + z v_{j mod n/2}
      z = zω
    endfor
  endif
end
```

10.3 这里的分治方式

这段代码是更常见的 Cooley-Tukey 拆分形式：

先分别计算偶下标输入序列的 DFT
再计算奇下标输入序列的 DFT
最后用单位根合并

也就是：

$$\begin{aligned} A_{\text{even}} &= (a_0, a_2, \dots, a_{n-2}) \\ A_{\text{odd}} &= (a_1, a_3, \dots, a_{n-1}) \end{aligned}$$

然后：

$$\begin{aligned} u &= \text{DFT}(A_{\text{even}}) \\ v &= \text{DFT}(A_{\text{odd}}) \end{aligned}$$

合并：

$$b_j = u_{\{j \bmod n/2\}} + \omega^j v_{\{j \bmod n/2\}}$$

10.4 时间复杂度

课件给出递推式：

$$t(n) = 2t(n/2) + O(n)$$

解得：

$$t(n) = O(n \log n)$$

这就是 FFT 相比 DFT 的核心改进。

11. 11.1.4 串行 FFT 非递归算法

递归 FFT 容易理解，但实际实现常使用非递归蝶式流图。

11.1 $n = 2$ 的蝶式

第 16 页先给出 $n=2$ 的情况：

$$\begin{aligned} b_0 &= a_0 + a_1 \\ b_1 &= a_0 - a_1 \end{aligned}$$

矩阵形式是：

$$\begin{bmatrix} b_0 \\ b_1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix}$$

这就是最小的蝶式计算。

11.2 $n = 4$ 的蝶式

第 16 页继续给出 $n=4$ 的情况。

经过拆分和重排，可以得到：

$$\begin{aligned} b_0 &= (a_0 + a_2) + (a_1 + a_3) \\ b_2 &= (a_0 + a_2) - (a_1 + a_3) \\ b_1 &= (a_0 - a_2) + (a_1 - a_3)\omega \\ b_3 &= (a_0 - a_2) - (a_1 - a_3)\omega \end{aligned}$$

这说明 $n=4$ 的 DFT 可以由两层蝶式计算完成。

11.3 蝶式计算流图

第 17 页给出蝶式计算流图。

左图是最小蝶式：

输入： a_0, a_1
输出： $a_0+a_1, (a_0-a_1)\omega^0$

右图是 $n=4$ 的蝶式流图，由两个阶段组成。

每个阶段都由若干独立的蝶式组成，因此非常适合并行计算。

12. $n = 8$ 的非递归蝶式图

第 18 页展示 $n=8$ 的非递归蝶式计算流图。

12.1 输出下标的位序反

图中输出顺序是：

b0, b4, b2, b6, b1, b5, b3, b7

课件特别说明：

b_i 的下标确定方式是取行号的位序反。

例如第 3 行：

3 = (011)₂

位序反后：

(110)₂ = 6

所以第 3 行输出的是：

b6

12.2 位序反的意义

FFT 非递归实现中，经常需要进行 bit-reversal permutation。

也就是把二进制下标反转：

000 → 000
001 → 100
010 → 010
011 → 110
100 → 001
101 → 101
110 → 011
111 → 111

这解释了为什么 $n=8$ 的输出不是自然顺序。

12.3 权因子的确定

第 18 页还指出一个问题：

下行线结点处的权因子的确定问题

也就是每个蝶式下支路应该乘哪个 ω^k 。

在 FFT 图中，权因子不是随便放的，而是由当前层数、蝶式位置和输出下标共同决定。

13. 11.2 并行 FFT 算法

第 19 页开始进入并行 FFT。

课件讲两个模型上的并行 FFT：

11.2.1 SIMD-MC² 上的 FFT 算法
11.2.2 SIMD-BF 上的 FFT 算法

这里：

MC² 可以理解为二维 mesh-connected computer
BF 指 butterfly network，蝶形网络

两个算法都利用 FFT 的蝶式结构，但映射到不同网络上。

14. 11.2.1 SIMD-MC² 上的 FFT 算法

14.1 处理器布局

第 21 页给出 SIMD-MC² 的处理器布局。

假设有 n 个处理器，组成：

$$n^{1/2} \times n^{1/2}$$

的二维方阵。

处理器按照行主序编号：

```
P0, P1, P2, P3
P4, P5, P6, P7
P8, P9, P10, P11
P12, P13, P14, P15
```

第 21 页图 11.5 展示的是 $n=16$ 时的 4×4 处理器网格。

14.2 算法输入输出

第 22 页给出算法 11.3。

输入：

a_k 处于 P_k 中

输出：

b_k 处于 P_k 中

也就是每个处理器开始时保存一个输入元素，最后保存一个输出元素。

15. SIMD-MC² 上的 FFT 算法步骤

15.1 第一步：初始化

```
for k = 0 to n-1 par-do
  ck = ak
endfor
```

所有处理器并行把输入 a_k 复制到临时变量 c_k 。

15.2 第二步：蝶式迭代

算法从：

$$h = \log n - 1 \text{ down to } 0$$

逐层执行蝶式计算。

每一层中：

```
p = 2h
q = n / p
z = wap
```

满足条件的处理器执行蝶式：

```
if k mod p = k mod 2p then
    ck    = ck + c_{k+p} z^{r(k) mod q}
    c_{k+p} = ck - c_{k+p} z^{r(k) mod q}
endif
```

这里 $r(k)$ 表示 k 的位序反。

15.3 $n = 16$ 时的参数变化

第 22 页右侧给出 $n=16$ 时各层参数：

```
h = 3, p = 8, q = 2, z = w^8
h = 2, p = 4, q = 4, z = w^4
h = 1, p = 2, q = 8, z = w^2
h = 0, p = 1, q = 16, z = w
```

这正好对应 FFT 的 $\log n$ 层蝶式计算。

15.4 第三步：位序反重排

最后：

```
for k = 0 to n-1 par-do
    bk = c_{r(k)}
endfor
```

也就是通过位序反排列把结果放回正确输出位置。

16. SIMD-MC² 算法示例： $n = 4$

第 23 到 26 页详细展示 $n=4$ 的例子。

16.1 第一步：初始化

第 23 页：

```
c0 ← a0
c1 ← a1
c2 ← a2
c3 ← a3
```

处理器布局为一个 2×2 网格：

```
P0  P1
P2  P3
```

16.2 第一次迭代 $h = 1$

第 24 页：

```
h = 1
p = 2
q = 2
z = w^2
```

满足条件的处理器是：

```
P0 和 P1
```

它们分别计算：

```
P0:
c0 = c0 + (w^2)^0 c2 = a0 + a2
c2 = c0 - (w^2)^0 c2 = a0 - a2

P1:
c1 = c1 + (w^2)^0 c3 = a1 + a3
c3 = c1 - (w^2)^0 c3 = a1 - a3
```

这一步相当于先把偶数位置和奇数位置内部做蝶式。

16.3 第二次迭代 $h = 0$

第 25 页:

```
h = 0
p = 1
q = 4
z = w
```

满足条件的处理器是:

```
P0 和 P2
```

计算:

```
c0 = (a0+a2) + (a1+a3)
c1 = (a0+a2) - (a1+a3)

c2 = (a0-a2) + (a1-a3)w
c3 = (a0-a2) - (a1-a3)w
```

这就是 $n=4$ DFT 的结果, 只是输出顺序还需要重排。

16.4 第三步: 输出重排

第 26 页:

```
b0 = c0
b1 = c2
b2 = c1
b3 = c3
```

这正是位序反带来的输出调整。

17. SIMD-MC² 算法分析

17.1 计算时间

第 26 页给出:

```
t_comp = O(log n)
```

因为 FFT 有 $\log n$ 层, 每层蝶式计算可并行完成。

17.2 选路时间

课件指出选路只涉及:

```
(2.4) 蝶式通信
(3) 位序反重排
```

在二维 mesh 上:

(2.4): $O(n^{\{1/2\}})$
(3): $O(n^{\{1/2\}})$

所以总选路时间是：

$O(n^{\{1/2\}})$

17.3 总时间

当 n 较大时，选路时间主导：

$t(n) = O(n^{\{1/2\}})$

这说明：

在二维网格上，即使 FFT 计算本身只有 $O(\log n)$ ，通信路由仍可能把整体时间提高到 $O(\sqrt{n})$ 。

18. 11.2.2 SIMD-BF 上的 FFT 算法

SIMD-BF 是基于蝶形网络的 FFT 算法。

BF 网络的结构天然适合 FFT，因为 FFT 本身就是蝶式计算。

19. 蝶形网络结构

第 28 页讲蝶形网络。

19.1 处理器布局

设：

$n = 2^k$

蝶形网络有：

$k + 1$ 层

每层有：

n 个处理器

所以总处理器数为：

$n(1 + \log n)$

第 r 行第 i 列的处理器记为：

$P_{\{r,i\}}$

其中：

$i = (a_1, a_2, \dots, a_k)_2$

19.2 互连方式

课件给出连接规则：

$P_{\{r,i\}}$ 与上一层 $P_{\{r-1,i\}}$ 和 $P_{\{r-1,j\}}$ 相连

其中：

i 的第 r 位为 0
 j 与 i 仅在第 r 位不同

这正是蝶式计算中“同位连接 + 交叉连接”的网络实现。

19.3 权因子的计算

第 28 页还给出 BF 网络中权因子 ω 的指数计算方法。

在处理器 $P_{\{r,i\}}$ 中， ω 的指数为：

$j = \exp(r,i)$

其中：

$\exp(r,i) = (a_r, \dots, a_1, 0, \dots, 0)$

含义是：

取 i 的前 r 位，做位序反，然后后面补 $k-r$ 个 0

这个公式用于确定每个蝶式下支路应乘的权因子。

20. $n = 8$ 的 BF 网络示例

第 29 页给出 $n=8$ 的 BF 网络图。

20.1 网络层数

因为：

$n = 8 = 2^3$

所以：

$k = 3$

网络有：

$k+1 = 4$ 层

对应图中的：

行0，行1，行2，行3

每层有 8 个处理器，对应列：

000, 001, 010, 011, 100, 101, 110, 111

20.2 图中权因子

第 29 页图中每层边上标有：

$$\omega^0, \omega^1, \omega^2, \dots, \omega^7$$

这些权因子正是根据第 28 页的 $\text{exp}(r, i)$ 规则确定的。

20.3 图的意义

这个图说明：

BF 网络把 FFT 的每一层蝶式计算直接做成一层网络连接，所以每一层通信可以在 $O(1)$ 时间内完成。

这比二维 mesh 更适合 FFT。

21. SIMD-BF 上的 FFT 算法分析

第 30 页给出算法分析。

21.1 正确性

课件说：

算法的正确性可以用归纳法证明

直观理解是：

- 第 0 层保存原始输入；
- 每经过一层，完成一层蝶式组合；
- 经过 $\log n$ 层后，得到完整 FFT 输出。

21.2 时间分析

第 30 页给出：

第 (1) 步时间： $O(1)$

对于第 (2) 步：

(2.1) 和 (2.2) 的计算时间为 $O(1)$

如果假定：

$\omega^{\{\text{exp}(r, i)\}}$ 已经计算好

那么选路时间也是：

$O(1)$

因为 BF 网络中的连接正好对应蝶式连接。

共有：

$\log n$

层，所以：

$t(n) = O(\log n)$

21.3 处理器数和成本

课件给出：

$$p(n) = n(1 + \log n)$$

成本：

$$c(n) = O(n \log^2 n)$$

因为：

$$\begin{aligned} c(n) &= t(n) \cdot p(n) \\ &= O(\log n) \cdot O(n \log n) \\ &= O(n \log^2 n) \end{aligned}$$

21.4 加速比和效率

课件给出：

$$\begin{aligned} S_p(n) &= O(n) \\ E_p(n) &= O(1 / \log n) \end{aligned}$$

这里可以这样理解：

- 串行 FFT 时间是 $O(n \log n)$ ；
- BF 并行 FFT 时间是 $O(\log n)$ ；
- 所以加速比约为 $O(n)$ ；
- 但使用了 $O(n \log n)$ 个处理器，因此效率约为 $O(1/\log n)$ 。

课件评价：

本算法的综合指标是较好的

因为它用较多处理器换来了非常低的并行时间。

22. $\omega^{\{\exp(r,i)\}}$ 的计算

第 30 页最后说明权因子如何计算。

22.1 初始条件

初始时：

$P_{\{k,i\}}$ 读入 $\omega^{\{\exp(k,i)\}}$

其中：

$$k = \log n$$

22.2 递推计算

如果：

$P_{\{r+1,i\}}$ 已有 $\omega^{\{\exp(r+1,i)\}}$

则：

$$\begin{aligned} P_{\{r,i\}} \text{ 中的 } \omega^{\{\exp(r,i)\}} \\ = \omega^{\{2 \exp(r+1,i)\}} \end{aligned}$$

所以经过：

$$\log n$$

步，就可以计算出每个处理器所需的权因子。

23. SIMD-MC² 与 SIMD-BF 的对比

项目	SIMD-MC² 上的 FFT	SIMD-BF 上的 FFT
网络结构	二维网格	蝶形网络
处理器数	n	n(1+log n)
计算时间	O(log n)	O(log n)
路由/通信时间	O(√n)	O(log n) 总层数，每层 O(1)
总时间	O(√n)	O(log n)
成本	约 O(n√n)	O(n log² n)
特点	处理器少，但通信较慢	处理器多，网络匹配 FFT 结构

最关键的区别是：

MC² 的网络结构不是 FFT 的天然结构，所以通信成为瓶颈；
BF 网络直接匹配蝶式计算图，所以每层通信非常高效。

24. 本章核心总结

24.1 DFT 的核心

DFT 定义为：

$$b_j = \sum_{k=0}^{n-1} a_k \omega^{kj}$$

直接计算需要：

$$O(n^2)$$

24.2 FFT 的核心

FFT 利用单位根性质，把规模 `n` 的 DFT 拆成两个规模 `n/2` 的 DFT。

递推式：

$$t(n) = 2t(n/2) + O(n)$$

所以：

$$t(n) = O(n \log n)$$

24.3 蝶式计算的核心

基本蝶式是：

$$\begin{aligned} u &= x + y \\ v &= (x - y)\omega^k \end{aligned}$$

FFT 的整个计算就是很多层蝶式组合。

24.4 位序反的核心

非递归 FFT 中，输出或输入常常需要位序反排列。

例如 $n=8$ ：

行 3 = (011)₂
位序反 = (110)₂ = 6
所以行 3 输出 b6

24.5 并行 FFT 的核心

并行 FFT 的关键不是单纯把蝶式并行执行，而是：

蝶式之间如何通信？
处理器网络是否匹配蝶式连接？
位序反重排代价有多大？
权因子如何分配？

25. 复习重点

本章最应该掌握：

1. DFT 的定义和矩阵形式
2. DFT 顺序代码为什么是 $O(n^2)$
3. FFT 如何把 DFT 分解为两个 $n/2$ 规模问题
4. 单位根性质： $\omega^n=1$ ， $\omega^{\{n/2\}}=-1$ ， $\omega^{\sim}=\omega^2$
5. 偶数项和奇数项的递归推导
6. 蝶式计算图的含义
7. 非递归 FFT 中的位序反
8. SIMD-MC² 上 FFT 的算法流程和 $O(\sqrt{n})$ 总时间
9. SIMD-BF 上 FFT 的蝶形网络结构和 $O(\log n)$ 时间
10. MC² 与 BF 的区别：通用网格 vs 专用蝶形网络

一句话概括：

第十一章讲的是 FFT 如何利用单位根的周期性和对称性，把 DFT 从 $O(n^2)$ 降到 $O(n \log n)$ ；并行 FFT 进一步把蝶式计算映射到处理器网络上，其中蝶形网络因为天然匹配 FFT 结构，可以达到 $O(\log n)$ 的并行时间。